

On the vertex separation of unicyclic graphs

Hsin-Hung Chou*

Department of Information Management, Chang Jung Christian University, Taiwan

Ming-Tat Ko

Institute of Information Science, Academia Sinica, Taiwan

Chin-Wen Ho

Department of Computer Science and Information Engineering,

National Central University, Taiwan

Gen-Huey Chen

Department of Computer Science and Information Engineering,

National Taiwan University, Taipei, Taiwan

Abstract

In the article “Computing the vertex separation of unicyclic graphs”, Information and Computation 192, pp. 123–161, 2004, Ellis et al. proposed an $O(n \log n)$ algorithm for computing both the vertex separation and an optimal layout of a unicyclic graph with n vertices. Using the data structures label and label array, we improve the time complexity of their algorithm to $O(n)$.

1 Background

The *vertex separation* problem can be defined using *linear layouts*, see [1]. A *linear layout* of a graph $G = (V, E)$ is a bijection L from V to the set of integers $\{1, 2, \dots, |V|\}$. The *vertex separator* at vertex v_i with respect to a layout L , denoted by $V_L(i)$, is the set of vertices with indices less than or equal to i and with one neighbor having index greater than i , i.e., $V_L(i) = \{v \mid (v, w) \in E, L(v) \leq i, \text{ and } L(w) > i\}$. The *vertex separation* with respect to G and L , denoted by $vs_L(G)$, is defined as the maximum vertex separator size over all vertex in V , i.e., $vs_L(G) = \max\{|V_L(i)| \mid 1 \leq i \leq |V|\}$. The *vertex separation* of G , denoted by $vs(G)$, is defined as the minimum vertex separation over all possible linear layout L of G , i.e., $vs(G) = \min\{vs_L(G) \mid L \text{ is a linear layout of } G\}$. A linear layout L of G is *optimal* if $vs_L(G) = vs(G)$.

Recently the vertex separation problem has received more attention because it is equivalent to

several well known important problems such as the node-searching problem [2], the gate matrix layout problem [3], and the path-width problem [4], with applications in VLSI layout.

For some special classes of graphs, the above problems can be solved in polynomial time, such as unicyclic graphs [5], permutation graphs [6], k -starlike graphs [7], and partial k -trees [8] for a fixed $k \geq 1$. Especially, trees [9] and cographs [10] can be solved in linear time.

A *unicyclic graph* is a graph constructed from a tree with one extra edge [5]. It can be depicted as a cycle with a set of trees in which each cycle vertex is the root of a tree, referred to as a *constituent tree*. In [5], Ellis et al. proposed an $O(n \log n)$ -time algorithm for computing both the vertex separation and an optimal layout of a unicyclic graph with n vertices. Their algorithm comprises two phases.

In Phase 1, for a unicyclic graph U , the vertex separation, an optimal layout, and the *type* (defined in [5]) of each constituent tree are computed by the methods proposed in [1] and [9]. The methods also used to compute the vertex separation k and an optimal layout L of $T = U - e$, which is the tree yielded by removing some cycle edge e from U . Phase 1 takes $O(n)$ time. At its completion, we know $k \leq vs(U) \leq k + 1$.

In Phase 2, the goal is to determine whether $vs(U) = k$, or $vs(U) = k + 1$. The process is comprised of two functions *vs_uni* and *vs_reduced_uni*, see Appendix. The function *vs_uni* takes the unicyclic graph U and the integer $k = vs(T)$ as input. It classifies the unicyclic graph as one of the five cases according to the

*Corresponding author.
chouhh@mail.cju.edu.tw

Email:

number of constituent trees with maximum vertex separations and their types, and analyzes each case individually. If $vs(U) = k$, it also computes an optimal layout. If $vs(U) = k + 1$, then the layout L computed in Phase 1 for T is also an optimal layout for U . In the five cases, Case 1 may make a recursive call, but Cases 3, 4, and 5 may invoke the function *vs_reduced_uni*. The function *vs_reduced_uni* takes a unicyclic graph U with two degree 2 cycle vertices r_i and r_j and the integer k as input. It returns true if and only if there exists a k -conforming layout for U with respect to r_i and r_j (defined in [5]). As indicated in [5], the time complexity $O(n \log n)$ is due to the computation of function *vs_reduced_uni* and the recursive call in Case 1 of function *vs_uni*.

In this paper, we inherit their concepts and propose a new data structure, *label array*, to organize the labels and layout lists of the constituent trees. The data structures can be referred to and updated efficiently so that the complexity of Ellis' algorithm can be improved to $O(n)$.

2 Label and Label Array

We first recall some notations and definitions for trees given in [5]. In the following, "tree" always means "rooted tree". Let $T[x]$ denote the subtree rooted at x within the rooted tree T .

Definition 1 ([1]). *A vertex x is k -critical in a rooted tree T if and only if $vs(T[x]) = k$ and there are two children u and v of x such that $vs(T[u]) = vs(T[v]) = k$.*

Definition 2 ([1]). *A tree is k -critical if it contains a k -critical vertex.*

Let $T[u; v_1, v_2, \dots, v_i]$ denote the tree with root u from which the subtrees with roots v_1 through v_i have been removed. The label of a rooted tree is defined as follows.

Definition 3 ([1]). *For any rooted tree $T[u]$, the label of u , denoted by $\lambda(T[u])$, is a list of integers $(a_1, a_2, \dots, a_p)$, where $a_1 > a_2 > \dots > a_p \geq 0$, and each a_i is associated with a vertex v_i for $1 \leq i \leq p$ such that*

- $vs(T[u]) = a_1$.
- For $1 \leq i < p$, $vs(T[u; v_1, \dots, v_i]) = a_{i+1}$.
- For $1 \leq i < p$, v_i is an a_i -critical vertex in $T[u; v_1, \dots, v_{i-1}]$.

- $v_p = u$. If a_p is marked with a prime "′", then there is no a_p -critical vertex in $T[u; v_1, \dots, v_{p-1}]$; otherwise, v_p is an a_p -critical vertex in $T[u; v_1, \dots, v_{p-1}]$. In both cases $T[u; v_p] = T[u; u]$ is the empty tree.

In particular, if $|\lambda(T[u])| > 1$, then the label of $T[u; v_1] = T - T[v_1]$ is $(a_2, \dots, a_p)$.

Lemma 1 ([1]). (1) *The label of a tree T rooted at u with n vertices is unique and can be computed in $O(n)$ time.*

(2) *When $|\lambda(T[u])| > 1$, $\lambda(T[u; v_1][u])$ can be obtained in constant time by removing the first element from $\lambda(T[u])$.*

All rooted trees $T[u]$ are classified into four types, denoted by $\tau(T[u])$, depending on the location of u .

Definition 4 ([1]). *Let T be a tree rooted at a vertex u and let $k = vs(T)$. The four possible types of $T[u]$ are as follows:*

- *Type NC : u has no child v such that $vs(T[v]) = k$.*
- *Type NCb : u has one child v such that $vs(T[v]) = k$, and T is not k -critical.*
- *Type C : u is a k -critical vertex in T .*
- *Type Cb : T is k -critical and u is not a k -critical vertex.*

The type of a rooted tree can be obtained from the labels of its root and children as follows:

1. If $|\lambda(T[u])| > 1$, then $\tau(T[u]) = Cb$.
2. If $|\lambda(T[u])| = 1$ and without "′", then $\tau(T[u]) = C$.
3. If $|\lambda(T[u])| = 1$ and with "′",

if one child of u has the same label as u , then $\tau(T[u]) = NCb$;

otherwise, $\tau(T[u]) = NC$.

Lemma 2. *Let T be a tree rooted at u and $\lambda(T[u]) = (a_1, a_2, \dots, a_p)$ associated with vertices $v_1, v_2, \dots, v_p$. The sequence $(\tau_1, \tau_2, \dots, \tau_p)$, where $\tau_i = \tau(T[u; v_1, v_2, \dots, v_{i-1}])$ for $1 \leq i \leq p$, can be obtained in $O(n)$ time, where n is the vertex number of T .*

Proof. From the definition of types, $\tau_1 = \dots = \tau_{p-1} = Cb$ when $p > 1$; and τ_p can be determined from the labels of u 's children in $T[u; v_1, v_2, \dots, v_{i-1}]$ obtained during the computation of $\lambda(T[u])$, as described in [1]. Since $r \leq \log n$ and u has at most $O(n)$ children, the lemma follows. **Q.E.D.**

Let U be a unicyclic graph with n vertices and $r_1, r_2, \dots, r_t$ be a set of cycle vertices of U . Let T_i be the constituent tree of U rooted at r_i , $\lambda(T_i[r_i]) = (a_1^i, a_2^i, \dots, a_{l_i}^i)$ associated with vertices $v_1^i, v_2^i, \dots, v_{l_i}^i$, and types $\tau_1^i, \tau_2^i, \dots, \tau_{l_i}^i$ for $i = 1, \dots, t$. For each a_j^i , we generate a record $R_j^i = [a_j^i, \tau_j^i, v_j^i]$. We use a positive integer ptr^* to store the maximum vertex separation over all labels $\lambda(T_i[r_i])$, $i = 1, \dots, t$. The label array of constituent trees $T_i, i = 1, \dots, t$ is an array A in which each entry $A[p], 1 \leq p \leq ptr^*$, contains

1. all records R_j^i with $a_j^i = p$;
2. five integers $n_{ALL}^p, n_{NC}^p, n_{NCb}^p, n_C^p$, and n_{Cb}^p such that
 - (a) n_{ALL}^p is equal to the total number of records in $A[p]$,
 - (b) n_{NC}^p is equal to the total number of records with type NC in $A[p]$,
 - (c) n_{NCb}^p is equal to the total number of records with type NCb in $A[p]$,
 - (d) n_C^p is equal to the total number of records with type C in $A[p]$,
 - (e) n_{Cb}^p is equal to the total number of records with type Cb in $A[p]$;
3. a positive integer ptr^p stores the maximum index q such that $1 \leq q < p$ and $n_{ALL}^q > 0$.

For examples, let $\lambda_1 = (9, 8, 6, 4')$ associated with vertices $v_1^1, v_2^1, v_3^1, v_4^1$, and types Cb, Cb, Cb, NC ; $\lambda_2 = (8')$ associated with vertices v_1^2 , and types NCb ; $\lambda_3 = (7, 6)$ associated with vertices v_1^3, v_2^3 , and types Cb, C . Figure 1 illustrates the label array of $\Lambda = \{\lambda_1, \lambda_2, \lambda_3\}$.

If $T_i, i = 1, \dots, t$, are all the constituent trees of U , their label array is called the label array of U and denoted by A_U . Let T be a subgraph of U (in fact it is a tree) by removing some cycle vertices and their respective constituent trees. Without ambiguity, the label array of the constituent trees in T is referred to as the label array of T , denoted by A_T .

Lemma 3. *Let U be a unicyclic graph with n vertices.*

$A_\lambda[1]$						
$A_\lambda[2]$						
$A_\lambda[3]$						
$A_\lambda[4]$	→ [4, NC, v_4^1]					
$A_\lambda[5]$						
$A_\lambda[6]$	→ [6, Cb, v_3^1] → [6, C, v_2^3]					
$A_\lambda[7]$	→ [7, Cb, $v_1^3]$					
$A_\lambda[8]$	→ [8, Cb, v_2^1] → [8, NCb, $v_1^2]$					
$A_\lambda[9]$	→ [9, Cb, $v_1^1]$					

n_{ALL}^i	n_{NC}^i	n_{NCb}^i	n_C^i	n_{Cb}^i	ptr^i
1	1	0	0	0	0

2	0	0	1	1	4
1	0	0	0	1	6
2	0	1	0	1	7
1	0	0	0	1	8

Figure 1: The label array of $\Lambda = \{\lambda_1, \lambda_2, \lambda_3\}$.

- (1) A label array can be constructed in $O(n)$ time.
- (2) Let e be a cycle edge of U . If $vs(U - e) = k$ and U has one k -critical constituent tree T_c with the k -critical vertex $u (\neq r_c)$, then the label array of $U - T_c[u]$ can be obtained from that of U in constant time.

Proof. (1) Since each element of the labels corresponds to a unique vertex, the total number of elements is bounded by n . Thus, statement (1) follows.

(2) In this case, $vs(U - T_c[u]) < k$; and in $A_U[k]$, $n_{ALL}^k = n_{Cb}^k = 1$, and $n_{NC}^k = n_{NCb}^k = n_C^k = 0$. Thus, the label array of $U - T_c[u]$ can be obtained from A_U by setting $ptr^* = ptr^k$, i.e., removing the entries $A_U[j], j > ptr^k$. **Q.E.D.**

3 Linear Time Algorithm

We insert the appropriate data structures into the functions *main*, *vs_uni*, and *vs_reduced_uni*, and update them efficiently to implement a linear time algorithm.

Phase 1.

The computation of the labels of $T_i[r_i], i = 1, \dots, t$, and $(U - e)[x]$, where x is an end vertex of e , and of the label array of U is inserted. From Lemma 1-(1) and Lemma 3-(1), the computation takes $O(n)$ time.

Phase 2.

Upon calling *vs_uni*(U, k), with the label array of U , the case that U is in can be determined in constant time.

In Case 1, there is only one k -critical constituent tree, T_c , in U . Let T' be the tree by removing T_c from U . The computation in Case

1 can be shown to be equivalent to that of the following program segment.

Case 1: {One k -critical constituent tree}

if T_c is of type C **then**

if $vs(T') = k$

then return(false)

else return(true);

else if $vs(U - T[u] - e) < k - 1$

then return(true)

else return($vs_uni(U - T[u], k - 1)$);

If the type of T_c is C , it takes $O(n)$ time to compute $vs(T')$ and then terminates. If the type of T_c is Cb , the computation for updating the label of $U - e$ to that of $U - T[u] - e$ and the label array of U to that of $U - T[u]$ is inserted. From Lemma 1-(2) and Lemma 3-(2), the updating takes constant time. Thus, out of the recursive call, it takes only constant time to determine $vs(U - T[u] - e)$ and prepare the data structures for the recursive call.

In Case 2, the process obviously takes constant time, but in Cases 3, 4, and 5, it may invoke the function $vs_reduced_uni$. Recall that $vs_reduced_uni$ takes a unicyclic graph U with two degree 2 cycle vertices r_i and r_j and the integer k as input. In each of these cases, the input of $vs_reduced_uni$ is U' and $vs(U - e)$, where U' is the unicyclic graph yielded by removing $T_i - r_i$ and $T_j - r_j$ from U for some i and j . Notice that U' is a unicyclic graph with two degree 2 cycle vertices r_i and r_j . Let T_1 and T_2 be the two remaining trees after removing r_i and r_j from U' (one of them can be empty). Denote the vertices adjacent to r_i by a and c and those adjacent to r_j by b and d , where a and b are in T_1 , and c and d are in T_2 . Note that for the recursive invocations in $vs_reduced_uni$, r_i and r_j are fixed in each case.

In the function vs_uni , when r_i and r_j are known, before each invocation of $vs_reduced_uni$, the computation of the labels of $T_1[a], T_1[b], T_2[c]$, and $T_2[d]$ and of the label arrays of constituent trees in T_1 and T_2 respectively is inserted. By Lemma 1-(1) and Lemma 3-(1), the computation takes $O(n)$ time. In the following discussion of the function $vs_reduced_uni$, without loss of generality, assume that $vs(T_1) \geq vs(T_2)$. The discussion for the case $vs(T_1) \leq vs(T_2)$ is similar.

In the function $vs_reduced_uni$, with the label arrays of T_1 and T_2 , the case that U is in can be determined in constant time.

The only case in which function $vs_reduced_uni$ recursively calls itself is when $vs(T_1) = k - 1$, $T_1[a]$ and $T_1[b]$ are $(k - 1)$ -critical, and T_1 contains one constituent tree T_c of vertex separation $k - 1$ and of type Cb . Let u be the $(k - 1)$ -critical vertex, which is in T_c . In this case, the problem is reduced to invoking $vs_reduced_uni$ with input $U - T[u]$ and $k - 1$. So, in the function $vs_reduced_uni$, the computation of the labels of $(T_1 - T[u])[a]$ and $(T_1 - T[u])[b]$, and of the label array of $T_1 - T[u]$ is inserted before the recursive call. The label of $(T_1 - T[u])[a]$ (respectively, $(T_1 - T[u])[b]$) can be obtained from the label of $T_1[a]$ (respectively, $T_1[b]$) by removing its first element. The label array of $T_1 - T[u]$ can be obtained from that of T_1 , A_{T_1} , by setting $ptr^* = ptr^{k-1}$. From Lemma 1-(2) and Lemma 3-(2), the computation takes constant time.

Since there are at most $O(\log n)$ calls of $vs_reduced_uni$ and each call takes at most constant time, the time complexity of the invocation of $vs_reduced_uni$ in function vs_uni is bounded by $O(\log n)$. The time complexity of the invocation of vs_uni in function vs_uni itself is bounded by $O(n)$, since outside the recursive call it takes $O(n)$ time, and there are at most $O(\log n)$ invocations of vs_uni , each of which takes constant time.

Thus, together with the $O(n)$ time complexity of Phase 1, the total time complexity of the algorithm is $O(n)$.

References

- [1] Ellis, J. A. and Sudborough, I. H. and Turner, J. S., "The vertex separation and search number of a graph," *Information and Computation*, 113(1): 50-79, 1994.
- [2] Kirousis, Lefteris M. and Papadimitriou, Christos H., "Searching and pebbling," *Theoretical Computer Science*, 47(2): 205-218, 1986.
- [3] Möhring, Rolf H., "Graph problems related to gate matrix layout and PLA folding," *Computational Graph Theory*, 17-51, 1990.
- [4] N. Robertson and P.D. Seymour, "Graph minors I. excluding a forest," *Journal of Combinatorial Theory. Series B*, 39-61, 1983.
- [5] Ellis, J. A. and Markov, M., "Computing the vertex separation of unicyclic graphs," *Information and Computation*, 192: 123-161, 2004.

- [6] Bodlaender, Hans L. and Kloks, Ton and Kratsch, Dieter, “Treewidth and pathwidth of permutation graphs,” *SIAM Journal on Discrete Mathematics*, 8(4): 606–616, 1995.
- [7] Peng, S.-L. and Ko, M.-T. and Ho, C.-W. and Hsu, T.-S. and Tang, C.-Y., “Graph searching on some subclasses of chordal graphs,” *Algorithmica*, 27(3-4): 395–426, 2000.
- [8] Bodlaender, Hans L. and Kloks, Ton, “Efficient and constructive algorithms for the pathwidth and treewidth of graphs,” *Journal of Algorithms*, 21(2): 358–402, 1996.
- [9] Skodinis, K., “Construction of linear tree-layouts which are optimal with respect to vertex separation in linear time,” *Journal of Algorithms*, 47: 40–59, 2003.
- [10] Bodlaender, Hans L. and Möhring, Rolf H., “The pathwidth and treewidth of cographs,” *SIAM Journal on Discrete Mathematics*, 6(2): 181–188, 1993.

Appendix

program main;

For all constituent trees, compute their vertex separation and type;

For some cycle edge e chosen at random, compute the vertex separation k ; and a layout L for the tree $U - e$;

if vs_uni(U, k)

then output (k , the layout created by vs_uni)

else output ($k + 1, L$);

function vs_uni(U : unicyclic graph, k : integer) : Boolean;

{The function assumes that $vs(U - e) = k$. The cases are identical to those defined in [Section 5.4], as also is the notation}

Case 1: {One k -critical constituent tree}

if $vs(T') = k$ **then** return(false)

else if T_c is of type C **then** return(true)

else compute $vs(T_c - T[u])$;

if $vs(U - T[u] - e) < k - 1$ **then** return(true);

else return(vs_uni($U - T[u], k - 1$));

Case 2: {Three or more non-critical k -trees}

return(false);

Case 3: {Exactly two non-critical k -trees}

return(vs_reduced_uni(U', k));

Case 4: {Exactly one non-critical k -trees, q = number of $(k - 1)$ -trees, not type NC}

Case 4.1: $\{0 \leq q \leq 1\}$

return(true);

Case 4.2: $\{q = 2\}$

for all choices of tree T_j from among the two $(k - 1)$ -trees **do**

if vs_reduced_uni(U', k){where $U' = U$ minus the bodies of T_i and T_j }

then return(true);

return(false)

Case 4.3: $\{q = 3\}$

for all choices of two trees T_i and T_j from among the k -tree and the three $(k - 1)$ -trees **do**

if vs_reduced_uni(U', k){where $U' = U$ minus the bodies of T_i and T_j }

then return(true);

return(false)

Case 4.4: $\{q \geq 4\}$

return(false);

Case 5: {No k -trees, q = number of $(k - 1)$ -trees, not type NC}

Case 5.1: $\{1 \leq q \leq 2\}$ return(true);

Case 5.2: $\{q = 3\}$

```

for all choices of two trees  $T_i$  and  $T_j$  from among
the three  $(k - 1)$ -trees do
    if vs_reduced_uni( $U', k$ ){where  $U' = U$  minus
the bodies of  $T_i$  and  $T_j$ }
    then return(true);
return(false)

```

Case 5.3: $\{q = 4\}$

```

for all choices of two trees  $T_i$  and  $T_j$  from among
the four  $(k - 1)$ -trees do
    if vs_reduced_uni( $U', k$ ){where  $U' = U$  minus
the bodies of  $T_i$  and  $T_j$ }
    then return(true);
return(false)

```

Case 5.4: $\{q \geq 5\}$

```

return(false);

```

function vs_reduced_uni(U, k) : Boolean;

{The cases and the notation used are identical to those defined in [Section 4.4].
We assume that $vs(T_1) \geq vs(T_2)$ }

Case 1: $\{vs(T_1) = k\}$

```

return(false);

```

Case 2: $\{vs(T_1) < k - 1\}$

```

return(true);

```

Case 3: $\{vs(T_1) = k - 1\}$

```

if  $T_1[a]$  is  $(k - 1)$ -critical and  $T_1[b]$  is  $(k - 1)$ -critical

```

```

then if  $T_1$  contains one  $k - 1$ , type Cb constituent

```

```

    then return(vs_reduced_uni( $U', k - 1$ ));

```

```

if  $T_1$  contains two  $(k - 1)$ , type NCb constituents

```

```

then return( $T_2$  contains only  $(k - 1)$ , type NC
constituents);

```

```

if  $T_1$  contains a  $(k - 1)$ , type C constituent

```

```

then return( $T_2$  contains no  $k - 1$  types other than
NC);

```

```

else { At least one of  $T_1[a], T_1[b]$  is not  $(k - 1)$ -critical }

```

```

    return( $T_1$  and  $T_2$  have complementary extensibili-
ties);

```